Workshop # 2

Multiple Linear Regression and Error Metrics

Instructions

For this task, start by choosing a **dataset** that contains both predictor variables **(features)** and a continuous target variable. This dataset will be useful not only for this lab but also for your assessment (see the assessment brief for more details).

Task 1: Data Selection:

a. Choose a dataset that has multiple features (predictor variables) and a continuous target variable.

- First, we need to load the dataset so we can work with it in Python.
- The dataset is assumed to be in a CSV file, a common file type for datasets. If yours is in another format (like Excel), use the appropriate pandas function (e.g., pd.read_excel() for Excel files).
- Replace "your_dataset.csv" with the path to your actual data file. For hint see the Figure below.

```
[ ]: import pandas as pd

     # Load your dataset into a DataFrame

     data = pd.read_csv("your_dataset.csv")  # Replace this with your file path
```

- Before choosing which columns to use, look at the data structure to see what features (input variables) and the target (output variable) are available.
- The .head() function displays the first few rows, helping you understand what's in each column.

```
[ ]:
     # Show the first five rows of the dataset to get a sense of the data

     print(data.head())

     # Display column names to identify which columns might be used as features and target

     print(data.columns)
```

b. **Hint:** Consider datasets like housing prices, car attributes (predicting price), or any dataset where the target is a numerical value. Datasets from platforms like Kaggle, UCI Machine Learning Repository, could be useful.

Task 2: Implement Multiple Linear Regression (print out actual values (y_test) and predicted values)

a. Use the **Multiple Linear Regression algorithm** from the **scikit-learn** library to predict the target variable from your chosen dataset.
b. Select Features and Target;
   - Features are the columns that your model will use to make predictions. As an example in a house price prediction dataset, features might include size, location, or number of rooms.
   - Target is the column you want to predict. In this case, it might be price.
   - Update the features and target based on what's available in your dataset. Replace ['**size**', '**bedrooms**', '**age**'] with your actual feature column names and price with your target column name. For hint see the picture below,

```
[ ]:  #Select columns to be used as features (X) and the target (y)

X = data[['size', 'bedrooms', 'age']]  # Replace with feature columns from your dataset
y = data['price']                       # Replace with your target column
```

   - For more clarity, see the figure below, you can modify this, as it is only for your understanding.

```
[ ]: from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

# Assume df is your DataFrame, and 'target' is the name of your target column

X = df.drop(columns=['target'])  # Features
y = df['target']                 # Target variable

# Split data into training and test sets

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)

# Initialize and fit the model

model = LinearRegression()
model.fit(X_train, y_train)
```

   - **Why Split?** Splitting the data into training and testing sets helps evaluate the model's performance on unseen data.
   - **How It Works**: The model learns from the X_train and y_train (training data), and then we test its performance using X_test and y_test (testing data).
   - **Example Split**: 80% for training, 20% for testing.

c. Make Predictions on the Test Set
   - After training, use the .predict() function on X_test to generate predictions.

- y_pred now contains the predicted values for each instance in X_test. The hint is shown in the figure below,

```
[ ]: # Make predictions on the test set

y_pred = model.predict(X_test)

# Print actual and predicted values

print("Actual values:", y_test.values)
print("Predicted values:", y_pred)
```

- **Hint:** model.predict(X_test) provides the predicted values. Printing both y_test and y_pred side by side helps in comparing them visually.

Task 3: Create four functions to print out the following metrics:

a. Mean Absolute Error

b. Mean Squared Error

c. Median Absolute Error

d. R^2 score - this is a function we have used from **scikit-learn** but you need to create your own function to do this.

a. Mean Absolute Error (MAE)

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

Hint: See the figure below to implement the function.

```
[ ]: def mean_absolute_error(y_true, y_pred):
        return sum(abs(y_true - y_pred)) / len(y_true)

# Usage
mae = mean_absolute_error(y_test.values, y_pred)
print("Mean Absolute Error:", mae)
```

b. Mean Squared Error (MSE)

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Hint: See the figure below to implement the function.

```
[ ]: def mean_squared_error(y_true, y_pred):
         return sum((y_true - y_pred) ** 2) / len(y_true)

     # Usage
     mse = mean_squared_error(y_test.values, y_pred)
     print("Mean Squared Error:", mse)
```


c. Median Absolute Error

$$\text{Median Absolute Error} = \text{median}(|y_i - \hat{y}_i|)$$

Hint: See the figure below to implement the function

```
[ ]: import numpy as np

     def median_absolute_error(y_true, y_pred):
         return np.median(abs(y_true - y_pred))

     # Usage
     medae = median_absolute_error(y_test.values, y_pred)
     print("Median Absolute Error:", medae)
```

d. R-Squared (R^2) Score

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

Hint: See the figure below to implement the function

```
[ ]: # R^2 Score

def r_squared(y_true, y_pred):

    ss_residual = sum((y_true - y_pred) ** 2)
    ss_total = sum((y_true - np.mean(y_true)) ** 2)

    return 1 - (ss_residual / ss_total)

# Usage

r2 = r_squared(y_test.values, y_pred)
print("R-squared:", r2)
```

Hint: Each function computes a specific error metric. Test these functions by passing in y_test.values and y_pred to ensure they're working as expected.

Note:

- **Research** the mathematical formulas for each error metric to ensure you understand how they quantify model performance. Once you understand the concept, translate it into code.
- **Python Programming Hint:** Refer to **scikit-learn** documentation on metrics like MAE, MSE, and R² to compare your functions to their implementations. Testing your functions against **sklearn.metrics.mean_absolute_error** or **sklearn.metrics.mean_squared_error** can help validate your custom implementations.
- Once you've completed all tasks, submit your **.py** file. If you have multiple files, compress them into a single **.zip** file.
- **Additional Step:** Include comments in your code to clarify the purpose of each function and any specific areas where you encountered difficulties.

Further Reading

For a deeper understanding, read **Chapter 4 of Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow**: Concepts, Tools, and Techniques to Build Intelligent Systems, which covers model evaluation and linear regression in detail.